

Programming II

Object Oriented Programming (OOP) **Abstract Class and Interfaces**

Dr/ Mahmoud Gamal

Abstract Classes:

- An abstract class cannot be instantiated, but other classes are derived from it.
- An Abstract class serves as a superclass for other classes.
- The abstract class represents the generic or abstract form of all the classes that are derived from it.
- A class becomes abstract when you place the abstract keyword in the class definition:

public abstract class(class name)....

Abstract Methods:

- An abstract method is a method that appears in a superclass, but expects to be overridden in a subclass.
- An abstract method has no body and must be overridden in a subclass.

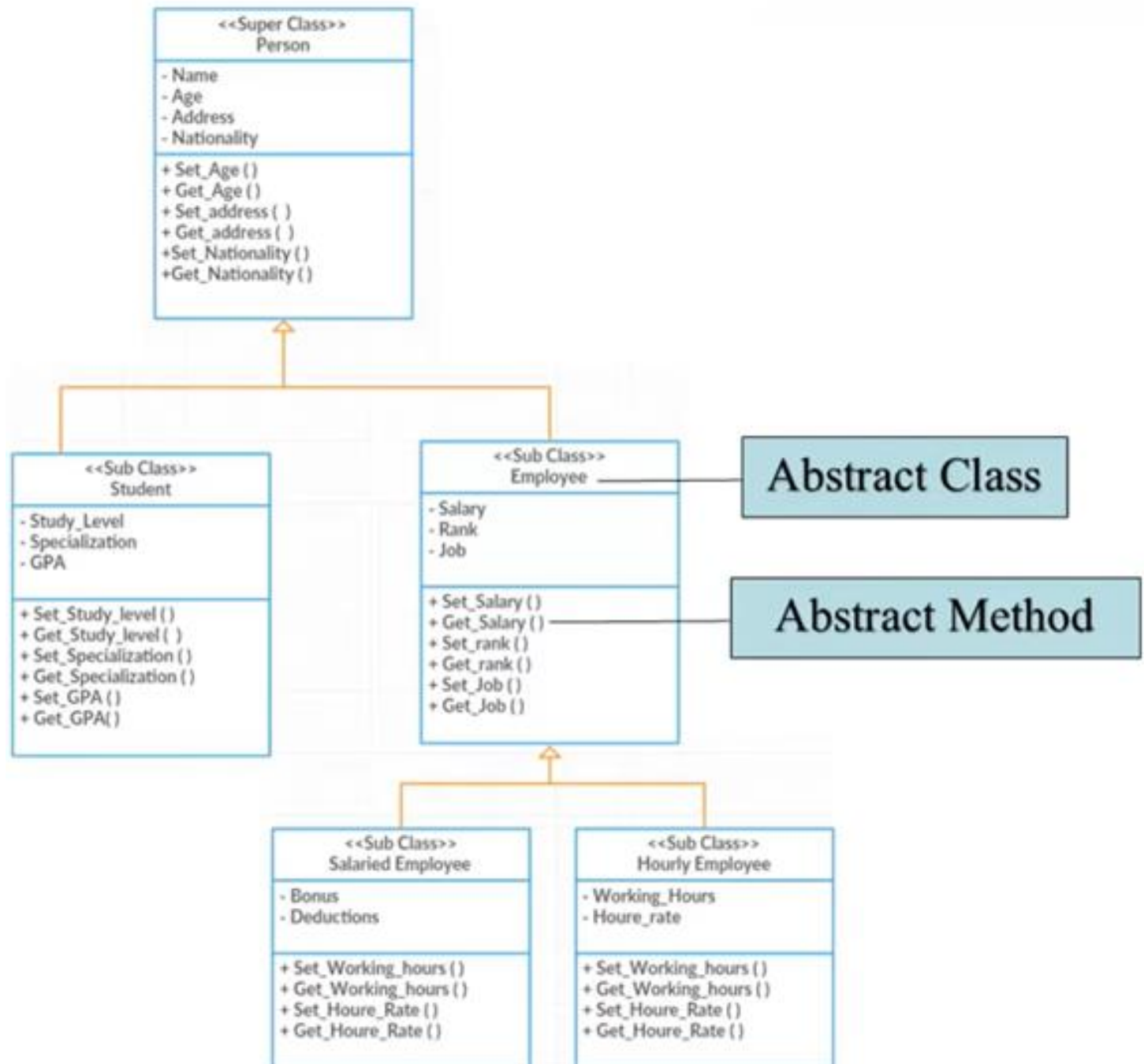
Access_specifier abstract return_type method_name (parameter list)

Ex: public abstract void get_salary();

(function header or function prototype)

- Abstract methods are used to ensure that a subclass implements the method.
- If a subclass fails to override an abstract method, a compiler error will result.

Example:



Abstract Class Employee:

```
public abstract class Employee extends person
{double salary;
  String rank;
  String Job;
public Employee()
{
}
public Employee (String n,double a , String ad, String nat, double sal, String r, String J)
{ super (n,a,ad,nat);
  salary = sal;
  rank = r;
  Job = J;
}
public void set_salary (double s)
{
  salary = s;
}

public abstract double get_salary ();  // In abstract method, write only method prototype
}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        Employee E1 = new Employee ();
    }
}
```

Output:

```
Main.java:12: error: Employee is abstract; cannot be instantiated
        Employee E1 = new Employee ();
                        ^
```

```
1 error
```

Salaried Employee Class:

```
public class Salaried_Employee extends Employee
{
    double bonous;
    double deduction;

    public Salaried_Employee()
    {
    }

    public Salaried_Employee (String n,double a , String ad, String nat, double sal, String r, String J, double b, double d)
    {
        super (n,a,ad,nat,sal,r,J);
        bonous = b;
        deduction = d;
    }
}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        Salaried_Employee sal1 = new Salaried_Employee ();
    }
}
```

Output:

```
Salaried_Employee.java:2: error: Salaried_Employee is not abstract and does not override abstract method get_salary() in Employee
public class Salaried_Employee extends Employee
      ^
1 error
```


Salaried Employee Class:

```
public class Salaried_Employee extends Employee
{
    double bonous;
    double deduction;

    public Salaried_Employee()
    {
    }

    public Salaried_Employee (String n,double a , String ad, String nat, double sal, String r, String J, double b, double d)
    {
        super (n,a,ad,nat,sal,r,J);
        bonous = b;
        deduction = d;
    }

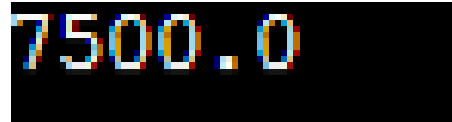
    @Override
    public double get_salary ()
    {
        return salary+bonous-deduction;
    }

}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        Salaried_Employee sal1 = new Salaried_Employee ("Ahmed", 20, "Alex", "Egyptian", 6000, "manager", "Engineer", 2000, 500);
        System.out.println(sal1.get_salary());
    }
}
```

Output:



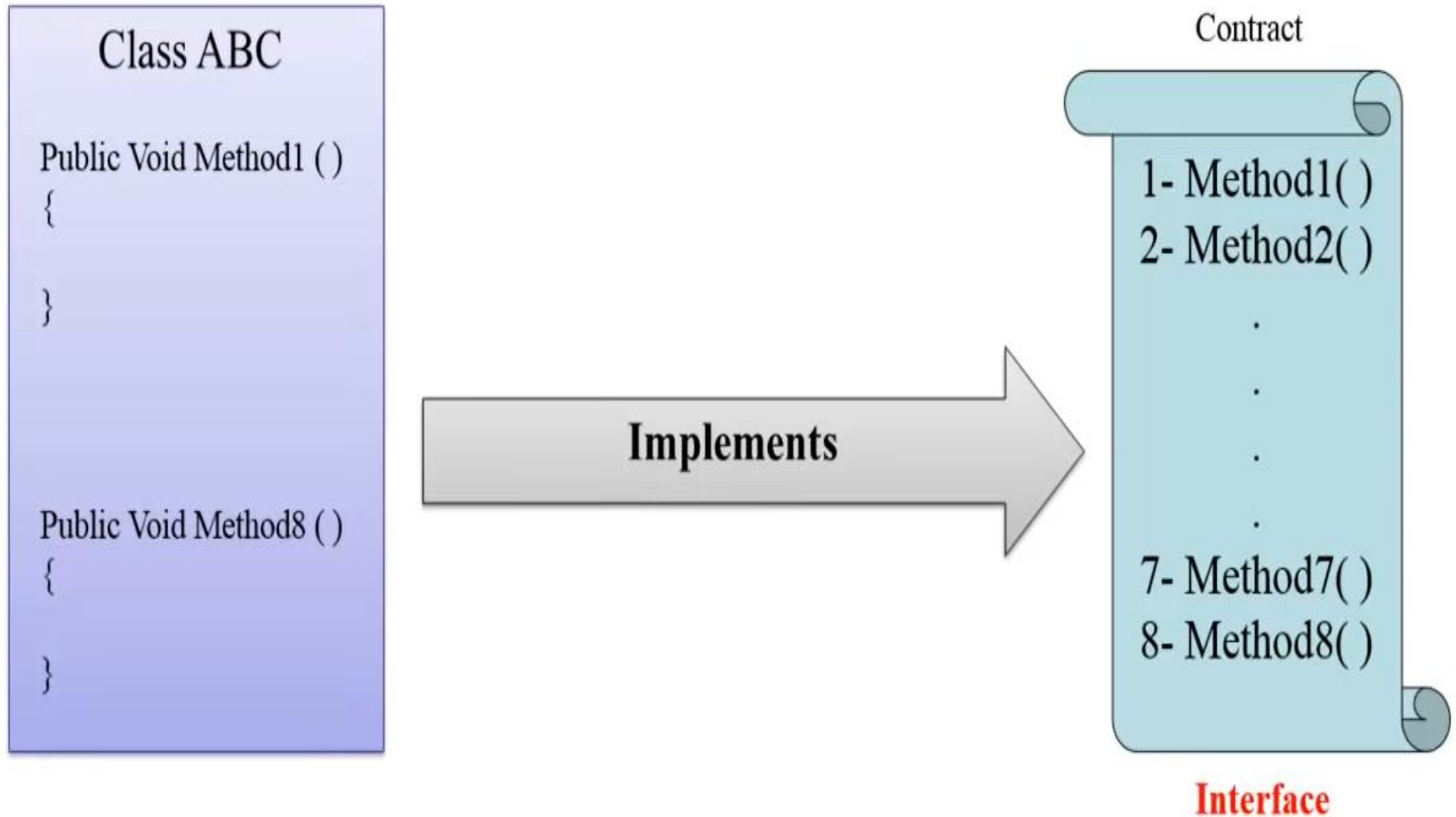
7500.0

Interfaces:

- An interface is similar to an abstract class that has all abstract methods.
- It cannot be instantiated, and all of the methods listed in an interface must be written elsewhere.
- The purpose of an interface is to specify behavior for other classes.
- It is often said that an interface is like a contract, and when a class implements an interface it must adhere to the contract.
- The general format of an interface definition:

```
public interface InterfaceName  
{  
    Method headers  
}
```

Interfaces:



Interfaces:

- If a class implements an interface, it uses the **implements** keyword in the class header.

```
public interface Retail_Item  
{  
    Method headers  
}
```

```
public class CD implements Retail_Item  
public class Book implements Retail_Item
```

Example:

Interface Retail Item:

```
public interface Retail_Item
{
    public double getitemprice();
}
```

Class CD:

```
public class CD implements Retail_Item
{
}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        Retail_Item r1 = new Retail_Item();
    }
}
```

Output:

```
Main.java:12: error: Retail_Item is abstract; cannot be instantiated
```

```
Retail_Item r1 = new Retail_Item();
```

```
^
```

```
1 error
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        CD cd = new CD();
    }
}
```

Output:

```
CD.java:1: error: CD is not abstract and does not override abstract method getitemprice() in Retail_Item
public class CD implements Retail_Item
      ^
1 error
```


Class CD:

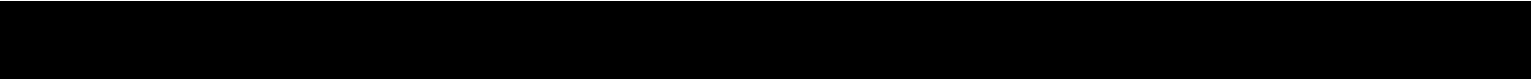
```
public class CD implements Retail_Item
{
    double price;
    String title;
    String artist;
    public CD()
    {
    }
    public CD(double price, String title, String artist)
    {
        this.price = price;
        this.title = title;
        this.artist = artist;
    }
    public double getitemprice()
    {
        return price;
    }
    public void setttitle(String title)
    {
        this.title = title;
    }
}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        CD cd = new CD();
        cd.getitemprice();
        Retail_Item r1 = new CD();
        r1.getitemprice();
    }
}
```

Output:

No error



Test Program:

```
public class Main
{
    public static void main(String[] args) {
        CD cd = new CD();
        cd.getitemprice();
        Retail_Item r1 = new CD();
        r1.getitemprice();
        r1.settitle("*****");
    }
}
```

Output:

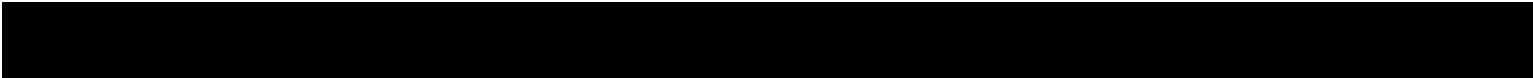
```
Main.java:16: error: cannot find symbol
        r1.settitle("*****");
            ^
    symbol:   method settitle(String)
    location: variable r1 of type Retail_Item
1 error
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        CD cd = new CD();
        cd.getitemprice();
        Retail_Item r1 = new CD();
        r1.getitemprice();
        cd.settitle("*****");
    }
}
```

Output:

No error



Implementing Multiple Interfaces:

- A class can be derived from only one superclass.
- Java allows a class to implement multiple interfaces.
- When a class implements multiple interfaces, it must provide the methods specified by all of them.
- To specify multiple interfaces in a class definition, simply list the name of interfaces separated by commas , after the implements keyword.

```
public class lecture implements Interface1, Interface2, Interface3
```

Example:

Retail Item interface:

```
public interface Retail_Item
{
    public double getitemprice();
}
```

Displayable interface:

```
public interface Displayable
{
    public void display();
}
```

CD Class:

```
public class CD implements Retail_Item, Displayable
{
    double price;
    String title;
    String artist;
    public CD()
    {
    }
    public CD(double price, String title, String artist)
    {
        this.price = price;
        this.title = title;
        this.artist = artist;
    }
    public double getitemprice()
    {
        return price;
    }
}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        CD cd = new CD();
    }
}
```

Output:

```
CD.java:2: error: CD is not abstract and does not override abstract method display() in Displayable
public class CD implements Retail_Item,Displayable
       ^
1 error
```


CD Class:

```
public class CD implements Retail_Item, Displayable
{
    double price;
    String title;
    String artist;
    public CD()
    {
    }
    public CD(double price, String title, String artist)
    {
        this.price = price;
        this.title = title;
        this.artist = artist;
    }
    public double getitemprice()
    {
        return price;
    }
    public void display ()
    {
        System.out.println("title = "+title+"price = "+price+"artist = "+artist);
    }
}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        CD cd = new CD();
    }
}
```

Output:

No error



Fields in Interfaces:

- An interface can contain field declarations.
- All fields in an interface are treated as **final and static**.
- Because they automatically become final, you must provide an initialization value.
- In this interface, FIELD1 and FIELD2 are final static int variables.

```
public interface Doable
{
    int FIELD1 = 1, FIELD2 = 2;
    (Method headers...)
}
```

- Any class that implements this interface has access to these variables.

Example:

Retail item interface:

```
public interface Retail_Item
{
    String storename = "ABC";
    public double getitemprice();
}
```

Displayable interface:

```
public interface Displayable
{
    public void display();
}
```

CD Class:

```
public class CD implements Retail_Item, Displayable
{
    double price;
    String title;
    String artist;
    public CD()
    {
    }
    public CD(double price, String title, String artist)
    {
        this.price = price;
        this.title = title;
        this.artist = artist;
    }
    public double getitemprice()
    {
        return price;
    }
    public void display ()
    {
        System.out.println("Store = "+storename+"\ntitle = "+title+"\nprice = "+price+"\nartist = "+artist);
    }
}
```

Test Program:

```
public class Main
{
    public static void main(String[] args) {
        CD cd = new CD();
        cd.display();
    }
}
```

Output:

```
Store = ABC
title = null
price = 0.0
artist = null
```